

# Supplementary Material: Prompt Templates

for *Component Attribution in Agentic Knowledge Graph Question Answering: A Controlled Ablation, Budget Analysis, and Failure Census*

Md. Sakif Khan

Sadia Sharmin

Seven templates constitute the whole of AGR’s interface to the language model, and six serve the baselines: one for the parametric control, one shared by the two static retrieval baselines, and four belonging to the reimplemented agentic baseline. All thirteen are reproduced here verbatim from the source that sends them, in the order a question encounters them, by a script that reads the code (`scripts/build_paper_supplement.py` in the repository named in the paper’s data-availability statement). No template carries a configuration constant, so one cached response serves every condition of a sweep; budgets are counters in code rather than instructions, and the evaluator’s remaining depth and backtracks are the only budget figures any prompt carries. Every call asks for JSON against a schema stated in the code beside the template.

## 1 Planner

`agr/planner.py`, constant `PLANNER_PROMPT`. Sent once per question unless the planner is removed. The first exemplar is the single-hop case that is not decomposed. Braces written `{like this}` are format fields filled at send time; doubled braces are literal.

```
You decompose questions into a minimal ordered chain of sub-objectives for navigating a
knowledge graph. Each sub-objective must be answerable by following one relation (possibly
through an event node). Use #N to reference the result of sub-objective N. Use as FEW sub-
objectives as possible.

For topic_mentions, extract ONLY specific named entities that appear literally in the question
(people, places, organizations, works, products). Do NOT extract generic type words such as "
celebrity", "university", "country", "senator", "airport", or "currency" -- these describe
what kind of answer is wanted, not where to start searching.

Example 1 (single hop -- do NOT decompose):
Question: "who is the president of France?"
{"sub_objectives": ["find the president of France"],
 "topic_mentions": ["France"]}

Example 2 (composition):
Question: "who directed the film that won Best Picture in 1998?"
{"sub_objectives": ["find the film that won Best Picture in 1998",
 "find the director of #1"],
 "topic_mentions": ["Best Picture"]}

Example 3 (conjunction):
Question: "which countries border France and have Spanish as official language?"
{"sub_objectives": ["find countries that border France",
 "find which of #1 have Spanish as an official language"],
 "topic_mentions": ["France", "Spanish"]}

Example 4 (superlative):
Question: "what is the largest city in the country where the Nile ends?"
{"sub_objectives": ["find the country where the Nile ends",
 "find the largest city in #1"],
```

```
"topic_mentions": ["Nile"]}  
  
Question: "{question}"
```

## 2 Relation scorer

agr/scorer.py, constant SCORE\_PROMPT. One batched call per explorer pass over the twenty candidates with the highest embedding score. Not sent when  $\alpha = 1$ . Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Sub-objective: {objective}  
  
Candidate knowledge-graph edges (anchor entity --relation-->):  
{candidates}  
  
For EACH candidate, rate from 0.0 to 1.0 how likely following that edge from  
that anchor leads toward completing the sub-objective. 1.0 = directly  
answers it, 0.0 = irrelevant. Judge the RELATION MEANING in context of the  
anchor, not surface word overlap.
```

## 3 Evaluator

agr/nodes.py, constant EVAL\_PROMPT. One call per explorer pass. The remaining depth and backtracks are substituted at send time; no other prompt carries a budget figure. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Current sub-objective: {objective}  
Recently retrieved facts:  
{facts}  
Remaining budget: {d_left} more expansions, {b_left} more backtracks.  
  
Decide:  
- "answer" if the facts complete the sub-objective (list satisfying entities in "resolved",  
exactly as named in the facts),  
- "continue" if this direction is promising but incomplete,  
- "backtrack" if this direction is a dead end.
```

## 4 Drafter and claim decomposer

agr/nodes.py, constant DRAFT\_PROMPT. One call when the search ends. Produces the prose, the answer entities, and the atomic claims the verifier checks. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}  
Facts retrieved from the knowledge graph:  
{facts}  
Entities identified as promising during exploration:  
{candidates}  
  
1. Draft an answer using ONLY the facts and candidates above. Use entity  
names exactly as written. If insufficient, say what could not be  
determined.  
2. List the answer entities themselves in "answer_entities", copied EXACTLY  
as they are named in the facts above -- these are the entities your draft  
asserts as the answer, not every entity it mentions. If your draft is a  
hedge ("could not be determined"), "answer_entities" MUST be [].  
IMPORTANT: "answer_entities" must be specific entities of the kind the
```

```
question asks for (a person for "who", a place for "where", etc.).
Never restate the question or name entity types/categories as the
answer. If you cannot name specific answer entities from the facts,
your draft must be a hedge and "answer_entities" must be [].
3. Decompose your draft into atomic factual claims, each as
{"h": entity name, "r": short relation phrase, "t": entity name}}.
Only include claims your draft actually asserts. A hedged draft has zero
claims.
```

## 5 Entailment check

agr/nodes.py, constant ENTAIL\_PROMPT. One batched call over the claims neither structural route settled. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Retrieved knowledge-graph triples:
{facts}

For each claim below, answer whether it is directly supported by the triples
above (true) or not (false). Unsupported includes: plausible but absent,
contradicted, or only inferable via outside knowledge.
Claims:
{claims}
```

## 6 Rewriter

agr/nodes.py, constant REWRITE\_PROMPT. One call on the give-up path only. The entity list it is given is already filtered; its own opinion about entities is discarded. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Draft answer: {draft}
The following claims in the draft are NOT supported by the knowledge graph:
{unsupported}
The following entities remain verified and MUST be kept in the answer,
named exactly as written: {kept}

Rewrite the draft: keep the verified entities and their supported claims,
and remove or explicitly hedge the unsupported claims. If the kept list is
empty, state that the answer could not be verified against the knowledge
graph. Respond with prose only.
```

## 7 Answerer (verifier skipped)

agr/nodes.py, constant ANSWER\_PROMPT. Reached only when the drafting call itself failed; no committed run took this path. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Facts retrieved from the knowledge graph:
{facts}

Entities identified as promising during exploration:
{candidates}

Answer using ONLY these facts and candidates. Give the answer entity name(s)
exactly as they appear above. If the information is insufficient, state what
```

```
could not be determined.
Also list in "answer_entities" the entities your answer
asserts (exactly as named above); [] if it asserts nothing.
```

## 8 No-retrieval control

agr/baselines/noretrieval.py, constant PROMPT. The single call of the parametric control. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Answer this question with the name(s) of the answer entity or
entities. Be concise. If you do not know, say so and return [].

Question: {question}
```

## 9 Vector-RAG and static GraphRAG

agr/baselines/vectorrag.py, constant PROMPT. Shared by the two static retrieval baselines, which differ only in what they retrieve. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Facts (retrieved from a knowledge graph):
{facts}

Question: {question}
Answer using ONLY the facts above. Name entities exactly as written in the
facts. If the facts are insufficient, say so and return [].
```

## 10 Think-on-Graph: relation pruning

agr/baselines/tog.py, constant REL\_PRUNE. One call per frontier entity per depth, over the first 40 relations returned. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Current entity: {entity}
Available relations (relation, direction, fanout):
{relations}
Select up to {w} relations most likely to lead to the answer.
Return {"relations": [{"rel": "...", "dir": "in|out"}]}.
```

## 11 Think-on-Graph: entity pruning

agr/baselines/tog.py, constant ENT\_PRUNE. One call per selected relation per depth, over the first 20 neighbours returned. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Candidate entities reached (via {entity} --{rel}-->):
{entities}
Select up to {w} entities most likely on the path to the answer.
Return {"entities": ["..."]}.
```

## 12 Think-on-Graph: sufficiency

agr/baselines/tog.py, constant SUFFICIENT. One call per depth, on the last sixty gathered triples. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Knowledge triples gathered so far:
{triples}
Can the question be answered from these triples alone?
Return {"sufficient": true|false}.
```

## 13 Think-on-Graph: answer

agr/baselines/tog.py, constant ANSWER. One call when the search stops, whether it finished or hit the cap. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Knowledge triples:
{triples}
Answer using ONLY these triples; name entities exactly as written.
If insufficient, say so and return [].
```